字串匹配詳解

32-E01 重疊、空模式與比較帳

推導

合法位移為0,1,2，每個窗口aaa都完整比較3字，總9次。它們可共用文字位置，定義並未禁止重疊。P=ε時長度0，依規格所有6個邊界0,⋯,5都是匹配；P長6>n=5時沒有長6窗口，輸出空。

結論與檢核

三種答案分別[0,1,2]、[0,1,2,3,4,5]、[]。先分支處理長度，才執行需要P[1]或長m首窗口的演算法，避免空模式與過長模式的越界。

32-E02 碰撞不等於匹配

推導

h=10、p=15 mod11=4。s=0窗口15，hash4，驗證成功。下一值 (10(4−1・10)+2) mod11=−58 mod11=8，對應52。再得 (10(8−5・10)+6) mod11=−414 mod11=4，對應26；hash相同，但15≠26，是假命中。再得 (10(4−2・10)+1) mod11=−159 mod11=6，對應61。最後 (10(6−6・10)+5) mod11=−535 mod11=4，對應15，驗證成功。

結論與檢核

hash序列[4,8,4,6,4]，正確位移[0,4]。省驗證會額外錯收2。每步由舊hash更新；直接重算各窗口mod11也同值，可作獨立數值核對。

32-E03 所有 Border 與建表

推導

π逐項為[0,0,1,0,1,2,3,2,3,4]。i=8讀b時，原k=3，P[4]=c不符，退π[3]=1，P[2]=b吻合，故π[8]=2；其他延伸逐項依字元相等決定。

P_10=abacababac的最長proper border為abac，長4；π[4]=0，所以所有proper border長度為4,0。若0<k<4也是border，因末4字正是abac，k同時必為abac的border，但其π[4]=0排除。若4<k<10則與「4是最長」矛盾。一般同一論證反覆用於π鏈，即得完整性。

結論與檢核

非空proper border只有abac。若錯設π[q]=q，fallback不減少，遇mismatch的while會永不終止，也違反proper的定義。

字串匹配詳解 （續）

32-E04 接受狀態也有轉移

推導

δ列q=0,1,2,3，欄a,b，分別為(1,0)、(1,2)、(3,0)、(1,2)。最後一列：abaa的最長模式前綴後綴為a，abab則為ab。

掃abababa的自動機狀態為1,2,3,2,3,2,3，輸出shift0,2,4。π(aba)=[0,0,1]；KMP在接受檢查時也是3，但輸出後保存1，接下一個b則延伸至2。因此KMP每字接受檢查時與自動機相同，接受後保存的狀態可不同。

結論與檢核

匹配後清零會在b處留0，再讀a才回1，漏掉shift2。正確fallback保留最後一個a，恰是下一次匹配的第一字。

32-E05 一個字元花很多比較

推導

模式前k字皆a，其π依序0,1,⋯,k−1，末字b的π為0。讀前k個a時q從0升至k。第k+1個a與P[k+1]=b不符，退到π[k]=k−1，再與P[k]=a吻合，回到k；其後每個a也同樣退一步再升一步。最後c使k→k−1→⋯→1→0，所有a/b比較均不符，保存0，單一字元有k次while迭代。

結論與檢核

q增加至多每字一次，且非負；每次fallback至少降低1，所以整段下降次數不超過全部增加量≤n。最後一字可Θ(k)而總計仍O(n)；兩種成本敘述並不矛盾。

32-E06 隨機 Base 的前提

推導

D(z)=Σ(X[j]−Y[j])z^(m−j)。至少一個係數在F_q非零，故D不是零多項式，次數≤m−1。因式定理與次數歸納給至多m−1根；b均勻取q個域元素，機率≤(m−1)/q（可再與1取小）。

若字元0與5都合法、q=5，長1的X=[0]、Y=[5]不同，但係數差−5在域中為0，D恆零，所有b都碰撞。此時聲稱機率≤0是錯誤的；缺的是字元編碼在F_q中的單射性。

結論與檢核

對每個非匹配窗口設I_s為碰撞指示，E[ΣI_s]=ΣE[I_s]，不要求事件獨立。輸入須在選base前固定，若觀察base後刻意選新字串，不能直接套固定字串的機率保證。

字串匹配詳解 （續）

32-E07 串流跨區塊

推導

預處理π=[0,1,0,1,2,2,3]；狀態只存q與全域已讀數i。第一塊後i=5、q=5。第二塊首a使q=6，接b使q=7，i=7，輸出i−m=0後q=3。再讀a,a,a,b，q依序4,5,6,7，i=11輸出4後q=3。

每次新字元的比較只讀模式、π及q。已讀文字對未來有用的相等資訊完全由最長可延伸border表示；正確性與逐字KMP相同，不依區塊切法。全域i跨區塊累加，輸出才是原文字位移。

結論與檢核

匹配[0,4]；儲存Θ(m)模式與π、O(1)個索引，不需保留舊文字。若題目另要回傳整段原文或上下文，則須額外緩衝，那是不同輸出規格。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

字串匹配詳解 （續）

原創驗算程式附錄: prefix_function.py

```
"""Build the KMP prefix function and scan for overlapping matches."""

def prefix_function(pattern):
    pi = [0] * len(pattern)
    for i in range(1, len(pattern)):
        j = pi[i - 1]
        while j and pattern[i] != pattern[j]:
            j = pi[j - 1]
        if pattern[i] == pattern[j]:
            j += 1
        pi[i] = j
    return pi


def matches(text, pattern):
    if not pattern:
        return list(range(len(text) + 1))
    if len(pattern) > len(text):
        return []
    pi = prefix_function(pattern)
    result, j = [], 0
    for i, char in enumerate(text):
        while j and char != pattern[j]:
            j = pi[j - 1]
        if char == pattern[j]:
            j += 1
        if j == len(pattern):
            result.append(i - len(pattern) + 1)
            j = pi[j - 1]
    return result


assert prefix_function("abab") == [0, 0, 1, 2]
assert matches("aaaa", "aa") == [0, 1, 2]
assert matches("", "") == [0]
assert matches("abc", "") == [0, 1, 2, 3]
assert matches("ab", "abcd") == []
print(matches("ababaca", "aba"))
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

字串匹配詳解 （續）

原創驗算程式附錄: string_match_lab.py

```
"""Reference executable models for the chapter; indices returned are zero-based."""
def naive(text,pattern):
    return [s for s in range(len(text)-len(pattern)+1) if text[s:s+len(pattern)]==pattern]

def prefix(pattern):
    pi=[0]*len(pattern); k=0
    for i in range(1,len(pattern)):
        while k and pattern[k]!=pattern[i]:k=pi[k-1]
        if pattern[k]==pattern[i]:k+=1
        pi[i]=k
    return pi

def kmp(text,pattern):
    if not pattern:return list(range(len(text)+1))
    pi=prefix(pattern);q=0;found=[];trace=[]
    for i,a in enumerate(text):
        chain=[q]
        while q and pattern[q]!=a:q=pi[q-1];chain.append(q)
        if pattern[q]==a:q+=1
        accepted=q==len(pattern)
        if accepted:found.append(i+1-len(pattern))
        trace.append((i+1,a,chain,q,accepted))
        if accepted:q=pi[q-1]
    return found,trace

def automaton(pattern,alphabet):
    if not pattern:return {(0,a):0 for a in alphabet}
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

字串匹配詳解 （續）

```
    pi=prefix(pattern);m=len(pattern);delta={}
    for a in alphabet:delta[0,a]=int(a==pattern[0])
    for q in range(1,m+1):
        for a in alphabet:
            delta[q,a]=q+1 if q<m and a==pattern[q] else delta[pi[q-1],a]
    return delta

def rabin_karp(digits,pattern,base=10,modulus=11):
    n=len(digits);m=len(pattern)
    if m==0:return list(range(n+1)),[]
    if m>n:return [],[]
    h=pow(base,m-1,modulus);p=t=0
    for a,b in zip(pattern,digits):p=(base*p+a)%modulus;t=(base*t+b)%modulus
    found=[];trace=[]
    for s in range(n-m+1):
        same=digits[s:s+m]==pattern
        trace.append((s,t,t==p,same))
        if t==p and same:found.append(s)
        if s<n-m:t=(base*(t-digits[s]*h)+digits[s+m])%modulus
    return found,trace

if __name__=='__main__':
    print('prefix',prefix('aabaaab'))
    print('KMP',kmp('aabaaabaaab','aabaaab'))
    print('RK',rabin_karp(list(map(int,'122312')),[1,2]))
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study